

Discovery Executive Summary

Project: dicoverly-123 · Generated: 24/07/2026, 17:52:40

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service/Repository layers (H2/H3), Domain Boundary Violations (H8), Shared DB Coupling (H9), and Dual-API duplication (H10).

EXECUTIVE SUMMARY

Klearcom is a modular monolith (Discovery IVR + Connect TFN) with a thin Laravel 12 API, a React 19 SPA, and a parallel Express `dev-api` that re-implements the same workflows. Controllers are short by LOC but fat by responsibility: Eloquent and KPI math live in HTTP handlers, Modules folders are documentation stubs only, and there are zero repository classes. The dominant risk is change amplification from duplicated business logic across Laravel, Express, and the SPA, plus shared Mongo collections and cross-domain dashboard/legacy reports that erase bounded-context ownership. Frontend has a usable `api/client` and React Query, but legacy class/widget patterns and hard-coded endpoint strings in pages still couple UI to backend shapes.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	---------	-------------	------	----------	-----------	----------	--------

H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC (Laravel avg); Express <code>server.js</code> 224 LOC	MODE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 handler methods with direct model/store access	HIGH
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	35+ Eloquent/store/Mongo access points; 0 repositories	HIGH
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (<code>LegacyDataMapper</code> , shared <code>buildTree</code> helpers)	MODE
H6	Direct SQL in Controllers	ORM/repository compliance %	>90%	60–90%	<60%	~38% of queries outside controllers	HIGH
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest <code>server.js</code> 224 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8 cross-domain access points	HIGH
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	37.5% (3/8 stores shared via Mongo <code>module</code>)	HIGH
H10	Dual API Duplication (additional)	Duplicated workflows across Laravel ↔ Express	0	1–3	>3	5+ workflows duplicated	HIGH
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC avg	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 components with hard-coded paths	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest ConnectPage 208 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels; small Zustand UI store	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (<code>LegacyMonitorPoller</code> , <code>LegacyDashboardWidget</code>)	MODE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Fat Controllers	Extract reachability/tree/KPI logic from <code>ConnectController</code> , <code>LegacyReportController</code> , and Express <code>server.js</code> into Application Services	MODERATE	HIGH
H2 Missing Service Layer	Create <code>DiscoveryJobService</code> , <code>ConnectMonitorService</code> , <code>DashboardKpiService</code> ; stop Eloquent/store use in handlers	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces for Eloquent models; bind in <code>AppServiceProvider</code> ; keep Mongo behind repository APIs	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace <code>LegacyDataMapper</code> <code>extract()</code> with DTOs; single <code>IvrTreeBuilder</code> domain service	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Enforce repository-only persistence; remove Eloquent from <code>Http/Controllers</code> and Mongo from Express routes	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Populate real <code>Modules/Discovery</code> & <code>Modules/Connect</code> ; move Reporting behind published DTOs/ACL	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Split Mongo collections per BC; document MariaDB ownership; ban cross-module writes	HIGH RISK	CRITICAL
H10 Dual API Duplication	Consolidate on Laravel as single API; proxy or delete Express <code>dev-api</code> duplication	HIGH RISK	CRITICAL
F5 Legacy Component Patterns	Convert <code>LegacyMonitorPoller</code> to hooks; add Error Boundaries; remove unused legacy widget	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers become thin HTTP adapters; Application/Domain Services own workflows and are reusable from jobs/CLI.
- Repository interfaces enable testing without MariaDB/Mongo and isolate schema churn to one layer.
- Real bounded contexts (Discovery, Connect, Reporting) stop silent cross-module coupling and support future extraction.
- Eliminating the dual Laravel/Express stack removes the largest source of divergent business rules.
- Frontend domain API modules plus purged legacy components keep the SPA aligned with a single backend contract.